

Обзор C++26

Обо мне



- Пишу на C++ больше 15 лет.
- Основал WG21 Russia в 2016 вместе с [@apolukhin](#).
- В 2016-2019 представлял предложения от РФ в комитете.
- Руководил разработкой поискового движка в Яндексе.
- Руководил инфраструктурой, поиском и ML в Озоне.

Этот доклад

1. Немного о том, что уже есть в C++26.
2. И о том, что мы на самом деле ждем от C++26.
3. Кратко о Pattern Matching.
4. Кратко о Рефлексии.
5. Много кода.
6. Много английских терминов и англицизмов.

Я прочитал обсуждаемые предложения в стандарт и присутствовал на обсуждениях в комитете, чтобы вам не пришлось этого делать.

Новости со встречи в Токио

- Чтение неинициализированной памяти теперь не является "неопределенным поведением" (UB). Появилось понятие "ошибочного поведения" (erroneous behavior).
- Началось обсуждение политик комитета.
 - Одобрili политику о `[[nodiscard]]` – не использовать `[[nodiscard]]` в стандарте.
 - Начали обсуждение политики о `noexcept`.
- `std::string + std::string_view`.
- `= delete("should have a reason");`
- ...и еще немного по мелочи.

Что уже есть в C++26

- `std::function_ref` и `std::copyable_function`.
- `pack...[indexing]`.
- `_` для неиспользуемых переменных.
- Библиотека линейной алгебры на основе BLAS.
- Поддержка идиомы RCU.
- `is_debugger_present()` и `breakpoint()`.
- ...и еще немного по мелочи.

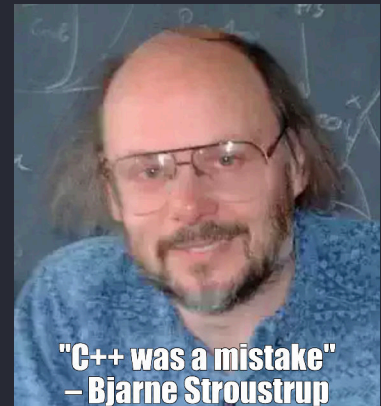
Что мы на самом деле ждем от C++26

	Статус	Консервативная оценка	Оптимистичная оценка
Senders	Рассмотрено в LWG	C++26	C++26
Networking	Зависит от Senders	C++29	C++26
Линейная алгебра	Замержено!	C++26	C++26
SIMD	Направлено в LWG	C++26	C++26
Контракты	Рассмотрено в SG21	C++29	C++26
Рефлексия	Направлено в EWG	C++26	C++26
Pattern Matching	Рассматривается в EWG	C++29	C++26

Что нужно обычным программистам

Наибольшую пользу для сообщества принесет pattern matching. Я проводил эксперименты с этим 15 лет назад. Мы не можем ожидать, что pattern matching просто появится в C++, люди должны работать над этим. И я хотел бы увидеть завершение работы над senders, это тянется уже три стандарта подряд. И после этого, возможно, статическую рефлексию.

— Бьёрн Страуструп,
последнее заседание комитета.



Pattern matching

Pattern matching: зачем?

```
1  using Expression = std::variant<AddNode, MulNode, double>;
2
3  double evaluate(const Expression &e) {
4      return std::visit<double>([<class T>(const T &arg) {
5          if constexpr (std::is_same_v<T, AddNode>)
6              return evaluate(*arg.l) + evaluate(*arg.r);
7          else if constexpr (std::is_same_v<T, MulNode>)
8              return evaluate(*arg.l) * evaluate(*arg.r);
9          else if constexpr (std::is_same_v<T, double>)
10             return arg;
11         else
12             static_assert(always_false_v<T>, "Non-exhaustive visitor!");
13     }, e);
14 }
```

C++ — ужасный язык.

— Линус Торвальдс, рассылка git.

Pattern matching: зачем?

```
1  using Expression = std::variant<AddNode, MulNode, double>;
2
3  double evaluate(const Expression &e) {
4      double value = 0.0;
5      if (std::holds_alternative<AddNode>(e))
6          value = evaluate(*std::get<AddNode>(e).l) + evaluate(*std::get<AddNode>(e).r);
7      else if (std::holds_alternative<MulNode>(e))
8          value = evaluate(*std::get<MulNode>(e).l) * evaluate(*std::get<MulNode>(e).r);
9      else if (std::holds_alternative<double>(e))
10         value = std::get<double>(e);
11     else
12         assert(false && "eeeh?");
13 }
```

Pattern matching: `std::variant`

```
1  using Expression = std::variant<AddNode, MulNode, double>;
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg ⇒ evaluate(*arg.l) + evaluate(*arg.r);
6          MulNode: let arg ⇒ evaluate(*arg.l) * evaluate(*arg.r);
7          double:  let arg ⇒ arg;
8          -       ⇒ throw EvaluationException(); // e.valueless_by_exception().
9      };
10 }
```

- Матчи вычисляются по порядку.
- После `⇒` можно использовать только выражения.
- Если матч не был найден, то вызывается `std::terminate()`.
- `let` используется для объявления binding'ов.

Pattern matching: `std::any`

```
1  using Expression = std::any;
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg => evaluate(arg.l) + evaluate(arg.r);
6          MulNode: let arg => evaluate(arg.l) * evaluate(arg.r);
7          double:  let arg => arg;
8          -       => throw EvaluationException(); // All other types.
9      };
10 }
```

Pattern matching: `std::any`

```
1  using Expression = std::any;
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg ⇒ evaluate(arg.l) + evaluate(arg.r);
6          MulNode: let arg ⇒ evaluate(arg.l) * evaluate(arg.r);
7          double:  let arg ⇒ arg;
8          -       ⇒ throw EvaluationException(); // All other types.
9      };
10 }
```

Вам не нужно беспокоиться о том, что не важно. В этом красота C++. Он позволяет вам сосредоточиться на том, что вы считаете важным.

— Бьёрн Страуструп, как нагаллюцинировал ChatGPT.

Pattern matching: как?

```
1  struct Expression : std::variant<AddNode, MulNode, double> {};  
2  
3  double evaluate(const Expression &e) {  
4      // return e match {  
5      switch (e.index()) {  
6          //      AddNode: let arg ⇒ evaluate(*arg.l) + evaluate(*arg.r);  
7          case 0: {  
8              auto &&arg = get<0>(e);  
9              return evaluate(*arg.l) + evaluate(*arg.r);  
10         }  
11         //      ...  
12         //      double: let arg ⇒ arg;  
13         case 2: {  
14             auto &&arg = get<2>(e);  
15             return arg;  
16         }  
17         //      _ ⇒ throw EvaluationException(); // e.valueless_by_exception().  
18         default:  
19             throw EvaluationException(); // e.valueless_by_exception().  
20     }  
21 }
```

Pattern matching: как?

```
1  using Expression = std::any;
2
3  double evaluate(const Expression &e) {
4      // return e match {
5      //     AddNode: let arg ⇒ evaluate(*arg.l) + evaluate(*arg.r);
6      if (auto *p = std::cast<Expression>::operator()<AddNode>(e)) {
7          auto &&arg = *p;
8          return evaluate(*arg.l) + evaluate(*arg.r);
9      //     ...
10     //     double: let arg ⇒ arg;
11     } else if (auto *p = std::cast<Expression>::operator()<double>(e)) {
12         auto &&arg = *p;
13         return arg;
14     //     _ ⇒ throw EvaluationException(); // e.valueless_by_exception().
15     } else {
16         throw EvaluationException(); // e.valueless_by_exception().
17     }
18 }
```

Pattern matching: полиморфные классы

```
1  struct Expression { virtual ~Expression() = default; };
2
3  double evaluate(const Expression &e) {
4      return e match {
5          AddNode: let arg => evaluate(*arg.l) + evaluate(*arg.r);
6          MulNode: let arg => evaluate(*arg.l) * evaluate(*arg.r);
7          ValNode: let arg => arg.v;
8          -       => throw EvaluationException();
9      };
10 }
```

- Внутри те же самые вызовы `std::cast`, который уже вызывает `dynamic_cast`.
- Есть техники, позволяющие выполнить этот код быстрее, чем цепочку вызовов `dynamic_cast`.

Pattern matching: switch

```
1  double evaluate(Operation op, double l, double r) {  
2      return op match {  
3          Operation::Add ⇒ l + r;  
4          Operation::Mul ⇒ l * r;  
5          Operation::Val ⇒ l;  
6          -              ⇒ throw EvaluationException();  
7      };  
8  }
```

- Метки являются константными выражениями.
- Матчи проверяются по порядку с помощью `operator==`.
- На деле компилируется в `switch`.

Pattern matching: лучше чем switch!

```
1  Color colorFromString(std::string_view s) {  
2      return s match {  
3          "red"    ⇒ Color::Red;  
4          "green"  ⇒ Color::Green;  
5          "blue"   ⇒ Color::Blue;  
6          "orange" ⇒ Color::Orange;  
7          _        ⇒ Color::Unknown;  
8      };  
9  }
```

- Помните, что матчи вычисляются по порядку.
- Есть возможности для оптимизации если позволить компилятору применить немного магии.

Pattern matching: `std::tuple`

```
1  double evaluate(const std::tuple<Operation, double, double> &op) {
2      return op match {
3          [Operation::Add, let l, let r] => l + r;
4          [Operation::Mul, let l, let r] => l * r;
5          [Operation::Val, let v, _]    => v;
6          -                             => throw EvaluationException();
7      };
8  }
```

- Код странный, исключительно для иллюстрации.
- Показывает, почему нам нужен `let`.

Pattern matching: structured bindings

```
1  double classify(Point point) {  
2      return point match {  
3          [0, 0] ⇒ PointClass::Origin;  
4          [0, _] ⇒ PointClass::YAxis;  
5          [_ , 0] ⇒ PointClass::XAxis;  
6          -      ⇒ PointClass::Other;  
7      };  
8  }
```

Паттерны можно компоновать!

```
1  using Expression = std::any;  
2  
3  double evaluate(const std::any &e) {  
4      return e match {  
5          AddNode: let [l, r] => evaluate(l) + evaluate(r);  
6          MulNode: let [l, r] => evaluate(l) * evaluate(r);  
7          double:  let v      => v;  
8          -        => throw EvaluationException();  
9      };  
10 }
```

- Также возможно:
 - let [[a, b], c].
 - [[0, 0], let c].
 - [MyPair: let [a, b], 0].

Pattern matching: использование `if`

```
1  int fib(int n) {  
2      return n match {  
3          let x if (x < 0) ⇒ 0;  
4          1 ⇒ n;  
5          2 ⇒ n;  
6          let x ⇒ fib(x - 1) + fib(x - 2);  
7      };  
8  }
```

```
1  if (expr match [let foo, 0]) {  
2      // `foo` is available here  
3  }
```

Pattern matching: точки кастомизации

- `operator=` для матчинга константных выражений.
- `std::tuple_size<T>`, `std::tuple_element<I, T>`, и `get<I>(v)` для типов, подобных `std::tuple`.
- `std::variant_size<T>`, `std::variant_alternative<I, T>`, и `index(v)` и `get<I>(v)` для типов, подобных `std::variant`.
- `std::cast` для `std::any`, `std::exception_ptr`, и полиморфных типов.

Pattern matching: открытые вопросы

- Матчинг для `std::variant<T, T>`.
- Матчинг для `std::expected<T, T>`.
- Вызов `std::terminate` если матч не был найден — это как-то чересчур...
 - Можем ли мы проверить во время компиляции, что `std::terminate` точно не будет вызван?
- Протоколы кастомизации. Протокол для типов, подобных `std::variant`, сейчас вызывает вопросы (например, `std::visit` не реализован в терминах существующего протокола).

Рефлексия

Рефлексия: немного истории

- Работа над предложением началась до появления `constexpr`, когда у нас было только метапрограммирование на шаблонах.
- Reflection TS с `reflexpr` уходит корнями в 2010-е.
- Люди успели навелосипедить больше чем влезает на один слайд:
 - `Boost.Describe`.
 - `Boost.PFR`.
 - `refl-cpp`.
 - ...
- Дизайн последней итерации предложения основан на `constexpr` функциях, которые появились только в C++20.

Рефлексия: базовые операции

```
1  constexpr std::meta::info r = ^int;  
2  [:r:] value = 42;  
3  
4  std::vector<[:r:]> v = { value };  
5  
6  [:^(vector<int>:)]::iterator it;
```

- `std::meta::info` — это непрозрачный тип для рефлексии.
- Префиксный оператор `^` выполняет рефлексию.
- `[:` и `:]` создают грамматические конструкции языка из объектов типа `std::meta::info`.

Рефлексия: базовые операции

```
1 constexpr std::meta::info v = ^std::vector;  
2 constexpr std::meta::info a = ^int;  
3 constexpr std::meta::info va = std::meta::substitute(v, {a});  
4  
5 [:va:] vec = {1, 2, 3}; // Got std::vector<int> back.
```

- Рефлексия работает для шаблонов.
- Есть ряд `constexpr` функций, работающих с `std::meta::info`.

Рефлексия: доступ к полям

```
1  struct Person {
2      std::string name;
3      std::string surname;
4  };
5
6  constexpr auto member_named(std::string_view name) {
7      for (std::meta::info field : nonstatic_data_members_of(^Person)) {
8          if (name_of(field) == name) return field;
9      }
10 }
11
12 int main() {
13     Person p;
14     p[:member_named("name")] = "John";
15     p[:member_named("surname")] = "Doe";
16     p[:member_named("nickname")] = "Coder"; // Error.
17 }
```

Рефлексия: кодогенерация с помощью `template for`

```
1  using StringHandler = std::string(*)(const void *);
2
3  template<class T>
4  static std::string stringize(const void *ptr) {
5      return std::to_string(*static_cast<const T *>(ptr));
6  }
7
8  constexpr std::array types = {^int, ^float, ^double}; // And maybe more.
9
10 constexpr std::array handlers = [] {
11     std::array<StringHandler, types.size()> result;
12     template for (std::size_t i = 0; constexpr auto e : types)
13         result[i++] = &stringize<typename[e]>;
14     return result;
15 }();
```

- Сегодня вы можете сделать то же самое с помощью макросов или списков типов.

Рефлексия: enum → std::string

```
1  template <typename E>
2      requires std::is_enum_v<E>
3  constexpr std::string enum_to_string(E value) {
4      template for (constexpr auto e : std::meta::enumerators_of(^E)) {
5          if (value == [:e:]) {
6              return std::string(std::meta::name_of(e));
7          }
8      }
9
10     return "<unnamed>";
11 }
12
13 enum Color { red, green, blue };
14 static_assert(enum_to_string(Color::red) == "red");
15 static_assert(enum_to_string(Color(42)) == "<unnamed>");
```

Рефлексия: `std::string` $\rightarrow$ `enum`

```
1  template <typename E>
2      requires std::is_enum_v<E>
3  constexpr std::optional<E> string_to_enum(std::string_view name) {
4      template for (constexpr auto e : std::meta::enumerators_of(^E)) {
5          if (name == std::meta::name_of(e)) {
6              return [:e:];
7          }
8      }
9
10     return std::nullopt;
11 }
```

Рефлексия: `std::string` → `enum`, но лучше

```
1  template<class E>
2      requires std::is_enum_v<E>
3  constexpr make_enum_pairs() {
4      constexpr auto size = std::meta::enumerators_of(^E).size();
5      std::array<std::pair<std::string_view, E>, size> result;
6      for (std::size_t i = 0; auto e : std::meta::enumerators_of(^E)) {
7          result[i++] = {name_of(e), value_of<E>(e)};
8      }
9      return result;
10 }
11
12 enum Color { red, green, blue };
13 constexpr auto mapping = frozen::make_unordered_map(make_enum_pairs<Color>());
```

Рефлексия: std::tuple

```
1  template<typename... Ts> struct Tuple {
2      struct storage;
3      [ :define_class(^storage, { data_member_spec(^Ts)... }):] data;
4      Tuple(): data{} {}
5      Tuple(Ts const& ...vs): data{ vs... } {}
6  };
7
8  template<std::size_t I, typename... Ts>
9  struct std::tuple_element<I, Tuple<Ts...>> {
10     static constexpr std::array types = {^Ts...};
11     using type = [ : types[I] :]; // Or you can do pack...[indexing].
12 };
13
14 constexpr std::meta::info get_nth_nsdm(std::meta::info r, std::size_t n) {
15     return nonstatic_data_members_of(r)[n];
16 }
17
18 template<std::size_t I, typename... Ts>
19 constexpr auto get(Tuple<Ts...> &t) noexcept -> std::tuple_element_t<I, Tuple<Ts...>& {
20     return t.data.[ : get_nth_nsdm(^decltype(t.data), I) :];
21 }
```

Рефлексия: интересности

- Рефлексия читает текущее состояние компилятора.
- Рефлексия изменяет текущее состояние компилятора.

```
1  constexpr auto type = ^std::vector<int>;
2
3  // This requires instantiation!
4  constexpr auto members = nonstatic_data_members_of(type);
5
6  // And we can check whether the type was instantiated...
7  constexpr bool fun = is_incomplete_type(type);
```

Вы можете создать счетчик (как макрос `__COUNTER__`) с помощью рефлексии. Ранее это можно было сделать через объявления, теперь это вызов `constexpr` функции.

Рефлексия: открытые вопросы

Что делает этот код?

```
1 constexpr auto tmpl = template_of(^int);
```

- Бросает исключение?
- Возвращает `std::expected`?
- Возвращает пустой `std::meta::info`?
- Какой-то другой механизм? Ошибка компиляции с "not-a-constant-expression"?

Было бы хорошо сделать обработку ошибок через исключения, но нужна поддержка исключений в `constexpr` контексте.

Рефлексия: открытые вопросы

- Текущее предложение не решает вопрос генерации кода.
 - Как сгенерировать `switch`? Актуальный вопрос для многих!
- API `define_class` в его текущей форме вызывает вопросы.
 - Как добавлять методы?
 - Как добавить `[[no_unique_address]]` к полю?
 - ...

Хотите узнать больше?

- [P2688R1](#): Pattern Matching: `match` Expression.
- [P2996R2](#): Reflection for C++26.
- [P2300R9](#): `std::execution`.
- [P1673R13](#): A free function linear algebra interface based on the BLAS.
- [P1928R8](#): `std::simd` — merge data-parallel types from the Parallelism TS 2.
- [P2900R6](#): Contracts for C++.

END